

# **Proyecto de Final Herramientas de Programación II**

## **Proyecto Final**

**Tecnología base:** C# Windows Forms, Base de Datos Relacional y GitHub

### **1. Presentación**

El proyecto final tiene como propósito que los estudiantes diseñen, desarrollen y sustenten un MVP funcional de una solución de software orientada a resolver una problemática real del contexto empresarial, aplicando los fundamentos trabajados en la asignatura.

El proyecto debe evidenciar competencias en Programación Orientada a Objetos, diseño de interfaces gráficas, conexión a bases de datos, documentación técnica, control de versiones y buenas prácticas de ingeniería de software.

No se espera una aplicación empresarial terminada a nivel industrial, pero sí un prototipo funcional, coherente, usable y técnicamente defendible, con potencial de uso real o comercial.

### **2. Objetivo general**

Desarrollar un MVP de software en C# Windows Forms, orientado a solucionar una necesidad del sector empresarial, integrando programación orientada a objetos, persistencia en base de datos, arquitectura en capas y buenas prácticas de desarrollo colaborativo.

### **3. Alcance del proyecto**

El proyecto debe corresponder a una solución empresarial, entregada en clase

Se podrán proponer otras ideas, siempre que: correspondan a un problema real, sean viables en C# Windows Forms, permitan aplicar POO, GUI, eventos y base de datos, y tengan alcance realista para un MVP académico.

### **4. Entregables obligatorios**

Cada equipo deberá entregar lo siguiente:

#### **4.5 Software funcional**

Aplicación desarrollada en C# Windows Forms, conectada a una base de datos relacional.

#### **4.6 Informe técnico adicional**

Documento que justifique la solución desde la perspectiva de la ingeniería de software.

#### **4.7 Repositorio GitHub**

Repositorio con evidencia del proceso real de desarrollo.

## Proyecto de Final Herramientas de Programación II

### 4.8 Manual de usuario

Documento breve con instrucciones de instalación, acceso y uso del sistema.

### 4.9 Sustentación

Presentación funcional y técnica del proyecto, con demostración del MVP.

## 5. Planeación por sprints

El desarrollo del proyecto se organizará en dos (2) sprints, cada uno con un valor equivalente al 20% de la nota final del curso.

### 5.1 Sprint 1 – Análisis, base de datos y FrontEnd (20%)

Este sprint se orienta a la comprensión del problema, definición de la solución, diseño de la base de datos y construcción inicial de la interfaz.

#### Productos esperados del Sprint 1:

a) definición del problema empresarial

#### **Sistema de control de inventario y abastecimiento**

Aplicación de escritorio para administrar productos, proveedores, entradas, salidas, existencias y niveles mínimos de stock en una empresa, almacén o bodega, permitiendo controlar la disponibilidad de productos, registrar movimientos de inventario y generar alertas de reabastecimiento cuando el stock sea insuficiente.

MVP debe cumplir con

- Inicio de sesión por roles: administrador, almacenista, supervisor.
- CRUD de productos, proveedores y movimientos de inventario.
- Registro de productos con código único, stock actual, stock mínimo y precio de venta.
- Registro de entradas y salidas de productos con actualización automática del stock
- Validaciones para no permitir stock negativo, cantidades menores o iguales a cero, códigos duplicados y fechas inválidas.
- Búsquedas por producto, proveedor, fecha y tipo de movimiento.
- Reportes de stock actual, productos con stock crítico e historial de movimientos
- Manejo de errores en formularios, disponibilidad de stock y conexión a BD
- Función diferencial: alertas automáticas de reabastecimiento cuando el stock baja del mínimo configurado

b) levantamiento y redacción de requisitos funcionales y no funcionales

## Proyecto de Final Herramientas de Programación II

| Requisitos Funcionales |                                    |                                        |                                                                                                                            |
|------------------------|------------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Código                 | Nombre del requisito               | Rol al que aplica                      | Descripción                                                                                                                |
| RF01                   | Inicio de sesión                   | Administrador, Almacenista, Supervisor | El sistema debe permitir el inicio de sesión mediante usuario y contraseña para controlar el acceso según el rol asignado. |
| RF02                   | Gestión de usuarios                | Administrador                          | El sistema debe permitir al administrador registrar, editar, eliminar y consultar usuarios del sistema.                    |
| RF03                   | Gestión de productos               | Administrador, Almacenista             | El sistema debe permitir registrar, consultar, actualizar y eliminar productos del inventario.                             |
| RF04                   | Código único de producto           | Administrador, Almacenista             | El sistema debe validar que cada producto tenga un código único y no permita registros duplicados.                         |
| RF05                   | Registro de stock                  | Administrador, Almacenista             | El sistema debe permitir almacenar el stock actual y el stock mínimo de cada producto.                                     |
| RF06                   | Registro de precio                 | Administrador                          | El sistema debe permitir registrar y actualizar el precio de venta de los productos.                                       |
| RF07                   | Gestión de proveedores             | Administrador, Almacenista             | El sistema debe permitir registrar, consultar, modificar y eliminar proveedores.                                           |
| RF08                   | Registro de entradas de inventario | Almacenista                            | El sistema debe permitir registrar entradas de productos al inventario aumentando automáticamente el stock disponible.     |
| RF09                   | Registro de salidas de inventario  | Almacenista                            | El sistema debe permitir registrar salidas de productos disminuyendo automáticamente el stock disponible.                  |
| RF10                   | Validación de stock negativo       | Sistema                                | El sistema no debe permitir realizar salidas de productos cuando la cantidad solicitada supere el stock disponible.        |

## Proyecto de Final Herramientas de Programación II

|      |                                  |                                        |                                                                                                                           |
|------|----------------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| RF11 | Validación de cantidades         | Sistema                                | El sistema debe validar que las cantidades ingresadas sean mayores a cero en cualquier movimiento de inventario.          |
| RF12 | Validación de fechas             | Sistema                                | El sistema debe validar que las fechas registradas en movimientos de inventario sean válidas.                             |
| RF13 | Búsqueda de productos            | Administrador, Almacenista, Supervisor | El sistema debe permitir buscar productos por nombre o código.                                                            |
| RF14 | Búsqueda de movimientos          | Administrador, Supervisor              | El sistema debe permitir consultar movimientos de inventario por producto, proveedor, fecha y tipo de movimiento.         |
| RF15 | Reporte de stock actual          | Administrador, Supervisor              | El sistema debe generar reportes con el stock actual de los productos registrados.                                        |
| RF16 | Reporte de stock crítico         | Administrador, Supervisor              | El sistema debe generar reportes de productos cuyo stock se encuentre por debajo del mínimo configurado.                  |
| RF17 | Historial de movimientos         | Administrador, Supervisor              | El sistema debe permitir consultar el historial de entradas y salidas de inventario realizadas.                           |
| RF18 | Alertas de reabastecimiento      | Administrador, Almacenista             | El sistema debe generar alertas automáticas cuando el stock de un producto sea menor o igual al stock mínimo establecido. |
| RF19 | Manejo de errores de formularios | Sistema                                | El sistema debe mostrar mensajes de error cuando existan campos vacíos, datos inválidos o errores en formularios.         |
| RF20 | Manejo de errores de conexión    | Sistema                                | El sistema debe notificar al usuario cuando exista un error de conexión con la base de datos.                             |

## Proyecto de Final Herramientas de Programación II

| Requisitos No Funcionales |                             |               |                                                                                                                |
|---------------------------|-----------------------------|---------------|----------------------------------------------------------------------------------------------------------------|
| Código                    | Nombre del requisito        | Categoría     | Descripción                                                                                                    |
| RNF01                     | Seguridad de acceso         | Seguridad     | El sistema debe requerir autenticación mediante usuario y contraseña para acceder a las funcionalidades.       |
| RNF02                     | Control por roles           | Seguridad     | El sistema debe restringir las funcionalidades según el rol del usuario autenticado.                           |
| RNF03                     | Disponibilidad del sistema  | Rendimiento   | El sistema debe estar disponible durante el horario operativo de la empresa.                                   |
| RNF04                     | Tiempo de respuesta         | Rendimiento   | El sistema debe responder las consultas y operaciones en un tiempo menor a 3 segundos.                         |
| RNF05                     | Integridad de datos         | Base de datos | El sistema debe garantizar la integridad y consistencia de la información almacenada en la base de datos.      |
| RNF06                     | Persistencia de información | Base de datos | Toda la información registrada debe almacenarse de manera permanente en una base de datos relacional.          |
| RNF07                     | Usabilidad                  | Interfaz      | La interfaz gráfica debe ser intuitiva, clara y fácil de utilizar para los usuarios.                           |
| RNF08                     | Manejo de errores           | Calidad       | El sistema debe mostrar mensajes claros y comprensibles cuando ocurra un error.                                |
| RNF09                     | Escalabilidad               | Arquitectura  | El sistema debe permitir futuras mejoras o integración de nuevos módulos sin afectar el funcionamiento actual. |
| RNF10                     | Mantenibilidad              | Desarrollo    | El sistema debe desarrollarse utilizando arquitectura en capas para facilitar el mantenimiento del código.     |
| RNF11                     | Compatibilidad              | Plataforma    | El sistema debe ejecutarse en equipos con sistema operativo Windows.                                           |

## Proyecto de Final Herramientas de Programación II

|       |                           |            |                                                                                                                    |
|-------|---------------------------|------------|--------------------------------------------------------------------------------------------------------------------|
| RNF12 | Tecnología de desarrollo  | Tecnología | El sistema debe desarrollarse utilizando C# Windows Forms y una base de datos relacional.                          |
| RNF13 | Respaldo de información   | Seguridad  | La base de datos debe permitir realizar copias de seguridad de la información registrada.                          |
| RNF14 | Validación de información | Calidad    | El sistema debe validar los datos ingresados antes de almacenarlos en la base de datos.                            |
| RNF15 | Confiabilidad             | Calidad    | El sistema debe garantizar el correcto registro de movimientos de inventario evitando inconsistencias en el stock. |

### c) identificación y documentación de reglas de negocio

| Reglas de Negocio |                                            |                                                                                              |                                                                                                            |
|-------------------|--------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Código            | Nombre de la regla                         | Descripción de la regla de negocio                                                           | Condición o validación                                                                                     |
| RN01              | Inicio de sesión obligatorio               | Todo usuario debe autenticarse mediante usuario y contraseña antes de acceder al sistema.    | No se permitirá el acceso sin credenciales válidas.                                                        |
| RN02              | Acceso según rol                           | Cada usuario solo podrá acceder a las funciones autorizadas según su rol dentro del sistema. | El administrador tendrá acceso total, mientras que almacenistas y supervisores tendrán permisos limitados. |
| RN03              | Código único de producto                   | Cada producto registrado en el sistema debe tener un código único.                           | No se permitirá registrar productos con códigos duplicados.                                                |
| RN04              | Registro obligatorio de datos del producto | Todo producto debe tener información básica obligatoria para ser registrado.                 | El producto debe incluir código, nombre, stock actual, stock mínimo y precio de venta.                     |

## Proyecto de Final Herramientas de Programación II

|      |                                         |                                                                                                                          |                                                                                             |
|------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| RN05 | Cantidades válidas en movimientos       | Las cantidades registradas en entradas y salidas de inventario deben ser mayores a cero.                                 | No se permitirán cantidades negativas ni iguales a cero.                                    |
| RN06 | Restricción de stock negativo           | El sistema no debe permitir salidas de productos que superen la cantidad disponible en inventario.                       | Si la cantidad solicitada es mayor al stock disponible, la operación será rechazada.        |
| RN07 | Actualización automática de stock       | Todo movimiento de inventario debe actualizar automáticamente las existencias del producto.                              | Las entradas aumentan el stock y las salidas disminuyen el stock.                           |
| RN08 | Registro obligatorio de movimientos     | Todo movimiento de inventario debe quedar almacenado en el historial del sistema.                                        | Cada movimiento debe registrar fecha, tipo de movimiento, usuario y cantidad.               |
| RN09 | Validación de fechas                    | Las fechas registradas en movimientos de inventario deben ser válidas.                                                   | No se permitirán fechas vacías o incorrectas.                                               |
| RN10 | Asociación de movimientos con productos | Todo movimiento de inventario debe estar relacionado con un producto existente.                                          | No se permitirá registrar movimientos sin un producto válido.                               |
| RN11 | Asociación de proveedores               | Toda entrada de inventario debe estar relacionada con un proveedor registrado.                                           | No se permitirán entradas sin proveedor asociado.                                           |
| RN12 | Consulta de información                 | Los usuarios autorizados podrán consultar información de productos y movimientos de inventario.                          | Las búsquedas podrán realizarse por producto, proveedor, fecha o tipo de movimiento.        |
| RN13 | Generación de reportes                  | El sistema debe generar reportes actualizados del inventario.                                                            | Los reportes incluirán stock actual, productos críticos e historial de movimientos.         |
| RN14 | Alerta de stock mínimo                  | El sistema debe generar una alerta automática cuando el stock de un producto sea igual o inferior al mínimo configurado. | Si el stock actual $\leq$ stock mínimo, el sistema mostrará una alerta de reabastecimiento. |

## Proyecto de Final Herramientas de Programación II

|      |                                       |                                                                                                 |                                                                                       |
|------|---------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| RN15 | Manejo de errores en formularios      | El sistema debe validar la información ingresada antes de procesarla.                           | Se mostrarán mensajes de error en caso de campos vacíos o datos inválidos.            |
| RN16 | Manejo de errores de conexión         | El sistema debe informar al usuario cuando exista un problema de conexión con la base de datos. | Se mostrará una notificación indicando el error de conexión.                          |
| RN17 | Eliminación controlada de información | Solo los usuarios autorizados podrán eliminar registros del sistema.                            | El administrador será el único rol con permisos de eliminación total.                 |
| RN18 | Persistencia de la información        | Toda la información registrada debe almacenarse de manera permanente en la base de datos.       | Los datos deben conservarse después del cierre del sistema.                           |
| RN19 | Registro de auditoría básica          | El sistema debe registrar qué usuario realizó cada movimiento de inventario.                    | Cada movimiento quedará asociado al usuario autenticado.                              |
| RN20 | Disponibilidad de productos           | El sistema debe reflejar en tiempo real la disponibilidad de productos en inventario.           | Toda actualización de stock debe visualizarse inmediatamente en consultas y reportes. |

### d) diseño preliminar de arquitectura en capas

#### Capa de Presentación

La capa de presentación corresponde a la interfaz gráfica desarrollada en C# Windows Forms. Esta capa será la encargada de la interacción directa con el usuario, permitiendo el ingreso de información y la visualización de datos del sistema.

#### Responsabilidades:

- Mostrar formularios y ventanas del sistema.
- Capturar datos ingresados por el usuario.
- Mostrar mensajes de validación y errores.
- Permitir la navegación entre módulos.
- Enviar solicitudes a la capa lógica.

#### Formularios principales:

## **Proyecto de Final Herramientas de Programación II**

- Inicio de sesión.
- Gestión de productos.
- Gestión de proveedores.
- Registro de entradas y salidas.
- Reportes de inventario.
- Alertas de stock crítico.

### **Capa de Lógica de Negocio**

La capa de lógica de negocio será responsable de procesar las reglas del sistema y validar las operaciones antes de acceder a la base de datos.

#### **Responsabilidades:**

- Validar reglas de negocio.
- Procesar entradas y salidas de inventario.
- Verificar disponibilidad de stock.
- Generar alertas automáticas de reabastecimiento.
- Controlar permisos según roles.
- Gestionar reportes e historial de movimientos.

#### **Ejemplos de validaciones:**

- No permitir stock negativo.
- Validar códigos únicos de productos.
- Validar cantidades mayores a cero.
- Validar fechas correctas.
- Capa de Acceso a Datos

### **La capa de acceso a datos**

será la encargada de la comunicación entre la aplicación y la base de datos relacional.

#### **Responsabilidades:**

- Realizar operaciones CRUD.
- Ejecutar consultas SQL.
- Gestionar conexiones a la base de datos.
- Recuperar y almacenar información.
- Manejar excepciones de conexión.

#### **Operaciones principales:**

- Insertar productos.
- Actualizar inventario.
- Registrar movimientos.
- Consultar reportes.
- Gestionar usuarios y proveedores.
- Capa de Base de Datos

## **Proyecto de Final Herramientas de Programación II**

### **La base de datos relacional**

almacenará toda la información del sistema de inventario.

#### **Responsabilidades:**

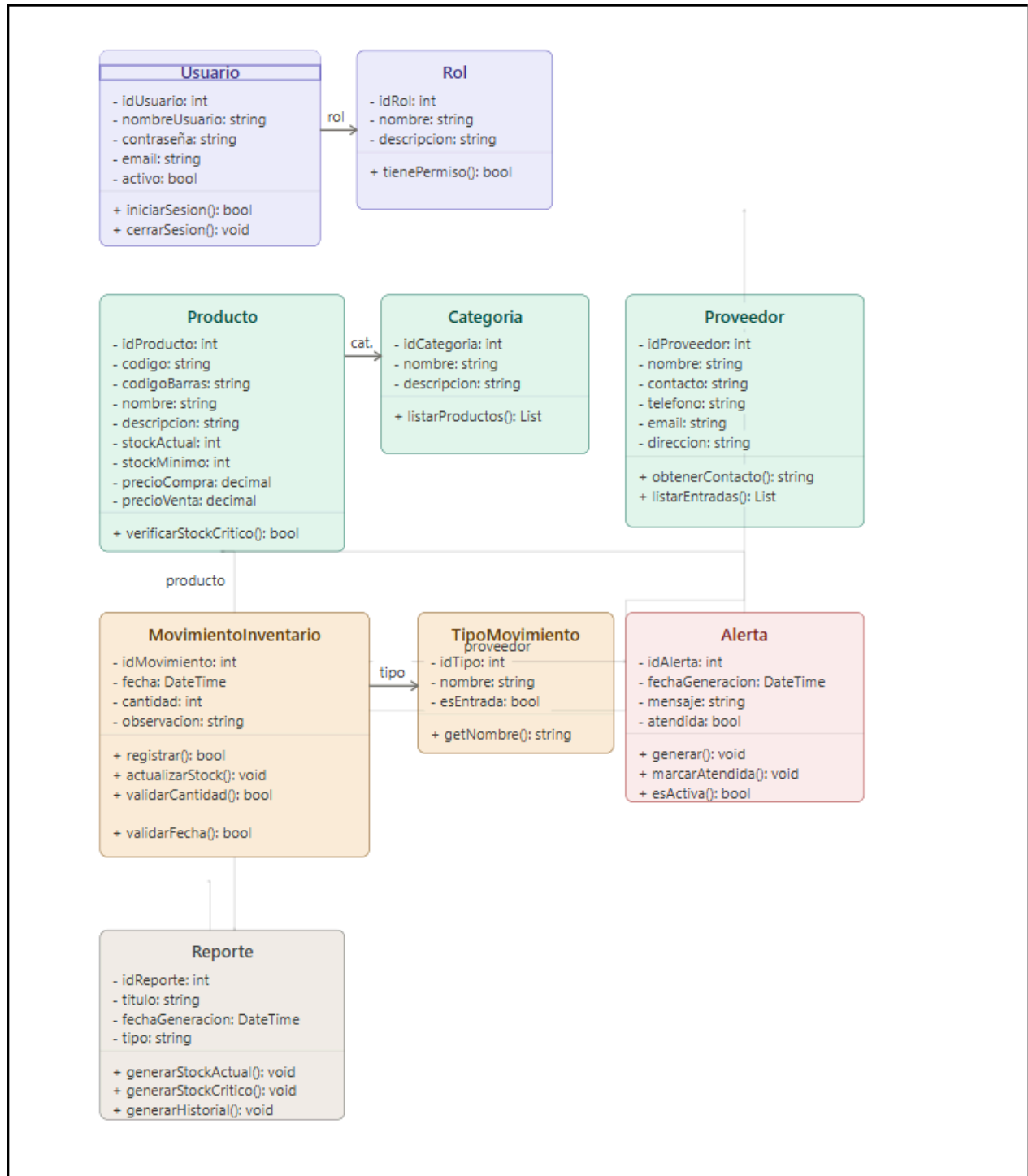
- Almacenar información persistente.
- Garantizar integridad de los datos.
- Mantener relaciones entre entidades.
- Permitir consultas y generación de reportes.

#### **Tablas principales:**

- Usuarios
- Roles
- Productos
- Proveedores
- MovimientosInventario
- TipoMovimiento
- Alertas
- Reportes

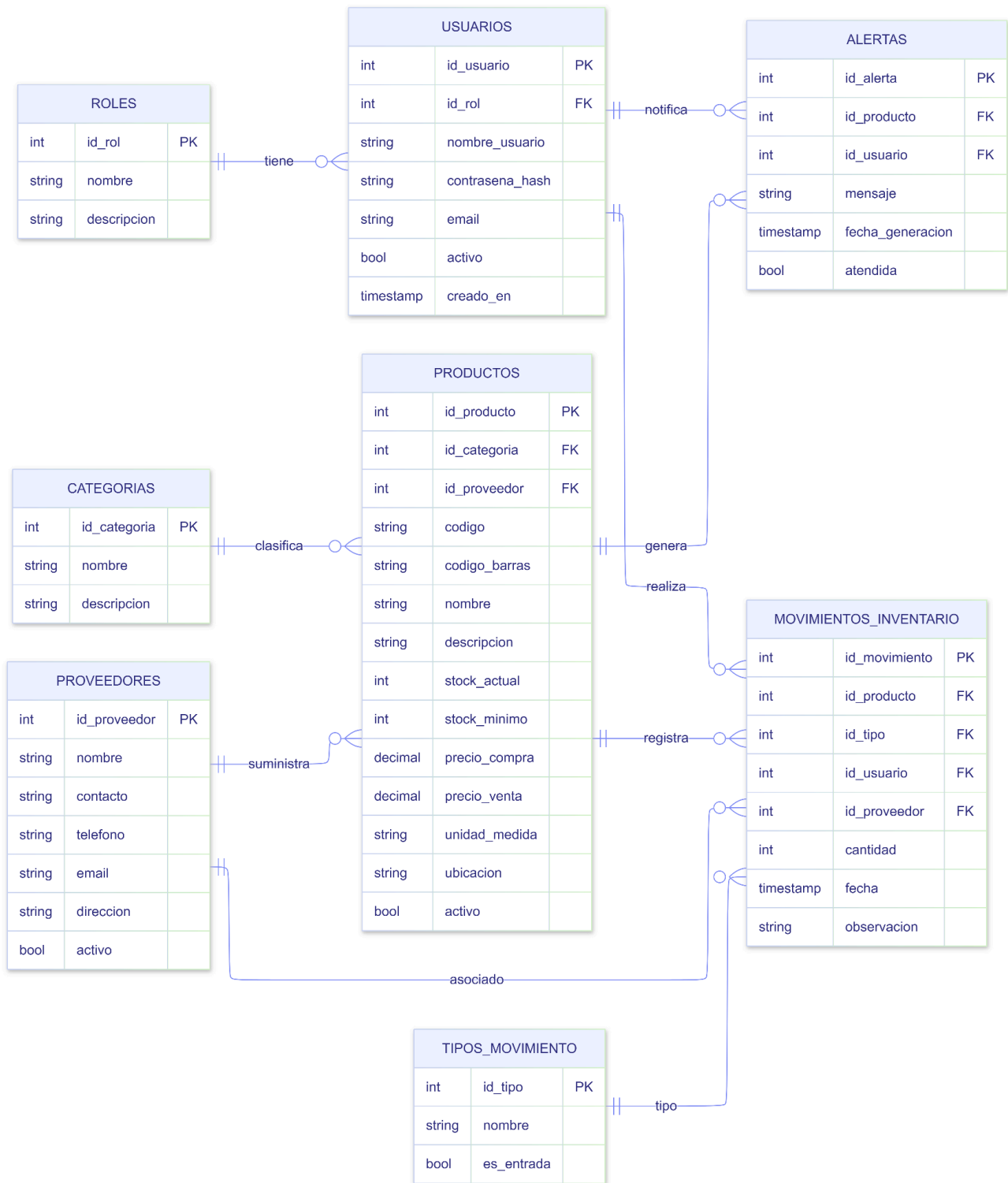
## Proyecto de Final Herramientas de Programación II

e) diagrama de clases preliminar



## Proyecto de Final Herramientas de Programación II

### f) modelo entidad-relación



## Proyecto de Final Herramientas de Programación II

### g) base de datos diseñada y normalizada

La base de datos del sistema fue diseñada bajo el modelo relacional utilizando **PostgreSQL**, aplicando principios de normalización hasta **la Tercera Forma Normal (3FN)**, con el objetivo de garantizar integridad, consistencia y evitar redundancia de datos.

El diseño tiene como objetivo garantizar:

- Integridad y consistencia de la información.
- Persistencia de datos.
- Escalabilidad del sistema.
- Facilidad de mantenimiento.
- Control de operaciones realizadas por los usuarios.
- Evitar redundancia e inconsistencias.

### Convenciones de nomenclatura

Para mantener un estándar uniforme dentro del sistema, se definieron las siguientes convenciones:

| Elemento        | Prefijo |
|-----------------|---------|
| Base de datos   | db_     |
| Tablas          | tb_     |
| Claves foráneas | fk_     |

Ejemplos:

- db\_inventario
- tb\_productos
- tb\_usuarios
- fk\_usuario\_rol

### Diseño de tablas

## Proyecto de Final Herramientas de Programación II

La base de datos está compuesta por las siguientes tablas principales:

| Tabla                     | Descripción                           |
|---------------------------|---------------------------------------|
| tb_rolés                  | Almacena los roles del sistema        |
| tb_usuarios               | Gestiona los usuarios autenticados    |
| tb_proveedores            | Registra información de proveedores   |
| tb_productos              | Contiene los productos del inventario |
| tb_tipo_movimiento        | Define los tipos de movimientos       |
| tb_movimientos_inventario | Registra entradas y salidas           |
| tb_alertas                | Almacena alertas de stock crítico     |

### Integridad de datos

El diseño implementa mecanismos de integridad mediante:

- Claves primarias (PRIMARY KEY).
- Claves foráneas (FOREIGN KEY).
- Restricciones UNIQUE.
- Restricciones CHECK.
- Valores por defecto mediante DEFAULT.

Ejemplos:

- No permitir stock negativo.
- No permitir cantidades menores o iguales a cero.
- Evitar códigos duplicados en productos.
- Mantener relaciones válidas entre entidades.

### Manejo de auditoría básica

Todas las tablas principales incluyen campos de auditoría para registrar información de creación y actualización de registros.

Campos implementados:

## Proyecto de Final Herramientas de Programación II

| Campo               | Descripción                      |
|---------------------|----------------------------------|
| creado_por          | Usuario que creó el registro     |
| fecha_creacion      | Fecha de creación                |
| actualizado_por     | Usuario que modificó el registro |
| fecha_actualizacion | Fecha de modificación            |

Esto permite llevar trazabilidad básica de las operaciones realizadas en el sistema.

### Uso de identificadores autoincrementables

Todas las tablas utilizan identificadores numéricos auto incrementables mediante:

GENERATED ALWAYS AS IDENTITY

Esto garantiza:

- unicidad de registros,
- facilidad de administración,
- mejor control de relaciones entre tablas.

### Relaciones principales

Las relaciones definidas en la base de datos son:

- Un usuario pertenece a un rol.
- Un movimiento de inventario pertenece a un producto.
- Un movimiento puede asociarse a un proveedor.
- Un movimiento es realizado por un usuario.
- Un movimiento tiene un tipo de movimiento.
- Una alerta pertenece a un producto.

### Normalización aplicada

La base de datos fue diseñada aplicando hasta la Tercera Forma Normal (3FN).

#### Primera Forma Normal (1FN)

Se eliminaron grupos repetitivos y se garantizaron atributos atómicos.

## Proyecto de Final Herramientas de Programación II

Ejemplo:

- Cada producto se almacenará con un único código.
- Cada movimiento se registrará con una única fecha.

### Segunda Forma Normal (2FN)

Todos los atributos dependen completamente de la clave primaria.

Ejemplo:

- Los datos de proveedores se almacenarán en una tabla independiente.
- Los roles se separarán de los usuarios.

### Tercera Forma Normal (3FN)

Se eliminaron dependencias transitivas y redundancias.

Ejemplo:

- Los tipos de movimiento se almacenan en una tabla separada.
- La información de roles no se duplica en usuarios.

### Motor de base de datos

El sistema utiliza **PostgreSQL** como sistema gestor de base de datos relacional debido a sus capacidades de integridad, seguridad y escalabilidad para aplicaciones empresariales.

Entre las principales características que justifican su uso se encuentran:

- Soporte de integridad referencial mediante claves primarias y foráneas.
- Alto nivel de seguridad y control de acceso.
- Estabilidad y confiabilidad en el manejo de información.
- Compatibilidad con el estándar SQL.
- Capacidad de escalabilidad para futuros módulos o crecimiento del sistema.
- Soporte para restricciones, validaciones y reglas de negocio avanzadas.
- Buen rendimiento en operaciones CRUD y consultas relacionales.
- Compatibilidad e integración con tecnologías de desarrollo como C#, Python, JavaScript y otros lenguajes utilizados tanto en aplicaciones de escritorio como web.
- Amplio soporte comunitario y documentación técnica.

h) script inicial de base de datos

## Proyecto de Final Herramientas de Programación II

```
-- =====
-- CREACION BASE DE DATOS
-- =====

CREATE DATABASE db_inventario;

-- Conectar me a la base de datos creada
\c db_inventario;

-- =====
-- TABLA ROLES
-- =====

CREATE TABLE tb_rols (

    id_rol INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    nombre VARCHAR(50) NOT NULL UNIQUE,

    creado_por INT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    actualizado_por INT,
    fecha_actualizacion TIMESTAMP
);

-- =====
-- TABLA USUARIOS
-- =====

CREATE TABLE tb_usuarios (

    id_usuario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    nombre VARCHAR(100) NOT NULL,

    usuario VARCHAR(50) NOT NULL UNIQUE,

    contrasena VARCHAR(255) NOT NULL,

    id_rol INT NOT NULL,
```

## Proyecto de Final Herramientas de Programación II

```
creado_por INT,  
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
actualizado_por INT,  
fecha_actualizacion TIMESTAMP,  
  
CONSTRAINT fk_usuario_rol  
    FOREIGN KEY (id_rol)  
    REFERENCES tb_rols(id_rol)  
);  
  
-- =====  
-- TABLA PROVEEDORES  
-- =====  
  
CREATE TABLE tb_proveedores (  
  
    id_proveedor INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  
    nombre VARCHAR(100) NOT NULL,  
  
    telefono VARCHAR(20),  
  
    correo VARCHAR(100),  
  
    direccion VARCHAR(200),  
  
    creado_por INT,  
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    actualizado_por INT,  
    fecha_actualizacion TIMESTAMP  
);  
  
-- =====  
-- TABLA PRODUCTOS  
-- =====  
  
CREATE TABLE tb_productos (  
  
    id_producto INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  
    codigo VARCHAR(50) NOT NULL UNIQUE,
```

## Proyecto de Final Herramientas de Programación II

```
nombre VARCHAR(100) NOT NULL,

descripcion TEXT,

stock_actual INT NOT NULL CHECK (stock_actual >= 0),

stock_minimo INT NOT NULL CHECK (stock_minimo >= 0),

precio_venta NUMERIC(10,2) NOT NULL CHECK (precio_venta >= 0),

creado_por INT,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

actualizado_por INT,
fecha_actualizacion TIMESTAMP
);

-- =====
-- TABLA TIPO MOVIMIENTO
-- =====

CREATE TABLE tb_tipo_movimiento (

    id_tipo_movimiento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    nombre VARCHAR(20) NOT NULL UNIQUE,

    creado_por INT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    actualizado_por INT,
    fecha_actualizacion TIMESTAMP
);

-- =====
-- TABLA MOVIMIENTOS INVENTARIO
-- =====

CREATE TABLE tb_movimientos_inventario (

    id_movimiento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

## Proyecto de Final Herramientas de Programación II

```
id_producto INT NOT NULL,

id_proveedor INT,

id_usuario INT NOT NULL,

id_tipo_movimiento INT NOT NULL,

cantidad INT NOT NULL CHECK (cantidad > 0),

fecha_movimiento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

observacion TEXT,

creado_por INT,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

actualizado_por INT,
fecha_actualizacion TIMESTAMP,

CONSTRAINT fk_movimiento_producto
    FOREIGN KEY (id_producto)
    REFERENCES tb_productos(id_producto),

CONSTRAINT fk_movimiento_proveedor
    FOREIGN KEY (id_proveedor)
    REFERENCES tb_proveedores(id_proveedor),

CONSTRAINT fk_movimiento_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES tb_usuarios(id_usuario),

CONSTRAINT fk_movimiento_tipo
    FOREIGN KEY (id_tipo_movimiento)
    REFERENCES tb_tipo_movimiento(id_tipo_movimiento)
);

-- =====
-- TABLA ALERTAS
-- =====

CREATE TABLE tb_alertas (
```

## Proyecto de Final Herramientas de Programación II

```
id_alerta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_producto INT NOT NULL,

mensaje TEXT NOT NULL,

estado VARCHAR(20),

fecha_alerta TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

creado_por INT,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

actualizado_por INT,
fecha_actualizacion TIMESTAMP,

CONSTRAINT fk_alerta_producto
    FOREIGN KEY (id_producto)
    REFERENCES tb_productos(id_producto)
);

-- =====
-- INSERTS INICIALES
-- =====

INSERT INTO tb_rols (
    nombre,
    creado_por
)
VALUES
('Administrador', 1),
('Almacenista', 1),
('Supervisor', 1);

INSERT INTO tb_tipo_movimiento (
    nombre,
    creado_por
)
VALUES
('Entrada', 1),
('Salida', 1);
```

## Proyecto de Final Herramientas de Programación II

### i) prototipo funcional del FrontEnd en Windows Forms

Se realizó el diseño inicial del FrontEnd en C# Windows Forms, desarrollando los formularios principales del sistema de inventario, incluyendo login, menú principal, gestión de productos, proveedores, movimientos y alertas.

El prototipo permite visualizar la estructura general y la interacción básica del sistema.

### j) navegación entre formularios

Se implementó la navegación básica entre formularios mediante botones y menús de acceso, permitiendo el desplazamiento entre los diferentes módulos del sistema.

La estructura de navegación propuesta facilita la interacción del usuario y la organización de funcionalidades.

### k) diseño visual coherente de la interfaz

La interfaz gráfica fue diseñada manteniendo consistencia visual entre formularios, utilizando estilos similares en colores, tipografías, tamaños y distribución de componentes.

El diseño busca ofrecer una experiencia visual clara, organizada e intuitiva para el usuario.

### l) repositorio GitHub con commits, ramas y README inicial

**Se evaluará:** análisis del problema, calidad de los requisitos funcionales y no funcionales, definición de reglas de negocio, modelo orientado a objetos preliminar, diagrama de clases preliminar, arquitectura propuesta, modelo entidad-relación, normalización inicial de base de datos, script inicial de BD, diseño y navegación del FrontEnd, organización inicial del repositorio GitHub, commits, ramas y README inicial